

Exception Handling Notes for Practical

1. What is an Exception?

An **exception** is an event that occurs during program execution and disrupts the normal flow of instructions. When an error occurs inside a method, the method creates an **exception object** and hands it to the Java Runtime (JVM). This is called **throwing an exception**. The JVM then searches the call stack for a block of code that can handle it (the **exception handler**). If none is found, the program terminates and the stack trace is printed.

Why handle exceptions? (a) To keep the program from terminating abnormally, (b) to separate error-handling code from normal logic, (c) to propagate errors up the call stack in a meaningful way, and (d) to release resources properly.

2. Exception Hierarchy

All exception classes descend from `java.lang.Throwable`. It has two direct subclasses: `Error` and `Exception`.

Type	Description	Examples
Error	Serious JVM / system problems. Should not be caught; not recoverable.	<code>OutOfMemoryError</code> , <code>StackOverflowError</code>
Checked exceptions	Subclasses of <code>Exception</code> (but not <code>RuntimeException</code>). Checked by the compiler ; must be caught or declared with <code>throws</code> .	<code>IOException</code> , <code>SQLException</code> , <code>ClassNotFoundException</code>
Unchecked exceptions	Subclasses of <code>RuntimeException</code> . Not checked at compile time; usually programming bugs.	<code>NullPointerException</code> , <code>ArithmeticException</code> , <code>ArrayIndexOutOfBoundsException</code>

3. Exception Handling Keywords

Keyword	Purpose
<code>try</code>	Encloses the code that may throw an exception. Must be followed by <code>catch</code> , <code>finally</code> or both (a <code>try-with-resources</code> may stand alone).
<code>catch</code>	Handles a specific type of exception thrown from the <code>try</code> block. Multiple <code>catch</code> blocks are allowed.
<code>finally</code>	Always executes (except <code>System.exit()</code> or JVM crash). Used for cleanup.
<code>throw</code>	Used to explicitly throw one exception object, e.g. <code>throw new MyException("msg");</code>
<code>throws</code>	Used in a method signature to declare checked exceptions the method may pass to its caller.

3.1 try-catch-finally – syntax and flow

```
try {  
    // risky code  
} catch (SpecificException e) {  
    // handler 1  
} catch (AnotherException e) {  
    // handler 2  
} finally {  
    // cleanup – always runs  
}
```

- If no exception occurs: try runs fully, catch is skipped, finally runs.
- If an exception occurs: the rest of try is skipped, the first **matching** catch runs, then finally runs.
- If no catch matches: finally runs, then the exception propagates to the caller.

3.2 Multiple catch blocks

Catch blocks are checked **top to bottom**. A subclass exception must be caught **before** its superclass; otherwise the compiler reports "exception has already been caught".

3.3 Multi-catch (Java 7+)

```
catch (InvalidAccountException | DailyLimitExceededException e) {  
    // same handling for both; e is implicitly final  
}
```

The alternatives in a multi-catch cannot be related by inheritance.

3.4 Nested try

A try block can contain another try-catch. If the inner block cannot handle the exception, it is passed to the outer block. This is useful when different parts of a task need different recovery strategies.

3.5 throw vs throws

	<code>throw</code>	<code>throws</code>
Purpose	Actually throws an exception object	Declares that a method may throw exceptions
Location	Inside a method body	In the method signature
Followed by	An instance (new X())	One or more class names, comma-separated
Count	One exception at a time	Several exceptions can be declared

	<code>`throw`</code>	<code>`throws`</code>
Checked?	Checked exceptions thrown must be handled or declared	Required for checked exceptions only

4. User-Defined (Custom) Exceptions

When built-in exceptions do not describe a business problem (e.g. "insufficient balance"), we create our own class by extending `Exception` (checked) or `RuntimeException` (unchecked).

```
class InsufficientBalanceException extends Exception {
    private final double shortfall;
    public InsufficientBalanceException(String msg, double shortfall) {
        super(msg);
        this.shortfall = shortfall;
    }
    public double getShortfall() { return shortfall; }
}
```

- **Extend ``Exception``** when the caller can reasonably recover and must be forced to handle the problem.
- **Extend ``RuntimeException``** for programming / validation errors (e.g. invalid argument).
- Provide constructors that accept a message and, ideally, a cause: `super(message, cause)`.
- Name the class with the suffix `Exception` and add useful fields (shortfall, missing course, etc.).

5. Exception Chaining

Exception chaining wraps a low-level exception inside a higher-level one so the original cause is not lost. Use the constructor `new Wrapper("msg", cause)` or `initCause()`. The original exception is later retrieved with `getCause()`.

```
catch (InvalidAccountException e) {
    throw new TransactionFailedException("Transfer failed", e);
}
// caller:
System.out.println(ex.getCause().getMessage());
```

6. try-with-resources (Java 7+)

Any object that implements `java.lang.AutoCloseable` can be declared in the `try ( ... )` header. Its `close()` method is called **automatically**, even if an exception occurs, so no explicit `finally` is needed for closing.

```
try (AuditLog log = new AuditLog("transfer")) {
    log.write("working...");
} // log.close() is called automatically
```

- Resources are closed in the **reverse order** of their creation.
- If both the try body and close() throw, the body's exception is the main one and the close() exception is attached as a **suppressed exception** (getSuppressed()).
- The catch and finally blocks of a try-with-resources run **after** the resources are closed.

7. The finally Block – Important Rules

- Executes whether or not an exception occurs and whether or not it is caught.
- Does **not** run if System.exit() is called inside try/catch, or the JVM crashes or the thread is killed.
- A return inside finally overrides any return or exception from try/catch – avoid this.
- Used for releasing resources such as files, sockets and database connections.

8. Rethrowing and Overriding Rules

- A catch block may rethrow the caught exception (throw e;) or throw a new one.
- An overriding method **cannot** throw a broader or new checked exception than the overridden method. It may throw narrower, unchecked, or none.
- A constructor can declare throws; a subclass constructor must also declare (or handle) the superclass constructor's checked exceptions.

9. Useful Methods of Throwable

Method	Use
getMessage()	Returns the detail message string.
getCause()	Returns the wrapped (original) exception or null.
printStackTrace()	Prints the exception and call stack to standard error.
getStackTrace()	Returns the stack as an array of StackTraceElement.
getSuppressed()	Returns exceptions suppressed during try-with-resources.
toString()	Returns class name + message.

10. Best Practices

- Catch the **most specific** exception first; avoid an empty catch block and avoid catching Throwable/Exception blindly.
- Never use exceptions for normal control flow.
- Always release resources (try-with-resources preferred).
- Preserve the cause when wrapping an exception.
- Throw early (validate arguments), catch late (where you can really recover).
- Document exceptions with @throws in Javadoc.
- Log the exception or rethrow it, but not both, to avoid duplicate logs.

11. Quick Revision Summary

Concept	One-line memory aid
Checked	Compiler forces you to handle or declare.
Unchecked	Compiler does not force; usually a bug.
throw	Throws an object. throws declares a possibility.
finally	Always runs unless the JVM stops.
Multi-catch	Handle unrelated exceptions with one block using .
Chaining	Wrap the old exception inside a new one; read it with getCause().
try-with-resources	Auto-close AutoCloseable objects in reverse order.